

Data Abstractions

Lecture 12

Overview

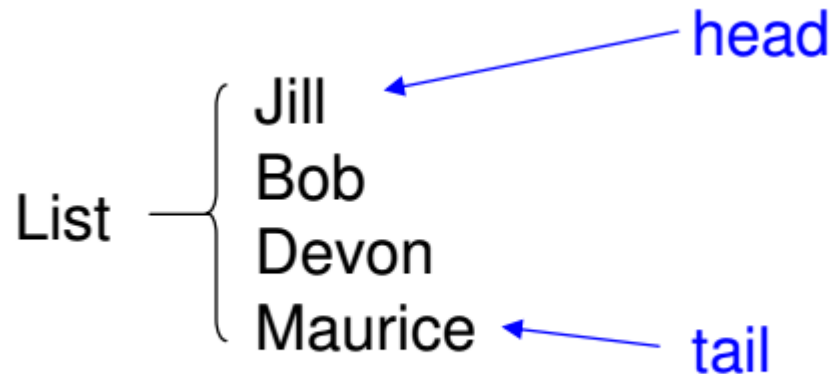
- Data Structure Fundamentals
- Implementing Data Structures
- Case Study Problem

Basic Data Structures

- We discussed in Programming Languages that **arrays** and **structures** are two aggregate data types directly supported by most programming languages
- Some other data structure:
 - List
 - Stack (Last-In, First-Out, LIFO)
 - Queue (First-In, First-Out, FIFO)
 - Tree
- We will discuss about efficient ways of implementing these data structures

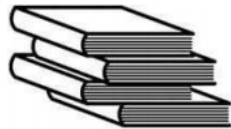
Terminology for Lists

- **List:** A collection of data whose entries are arranged sequentially
- **Head:** The beginning of the list
- **Tail:** The end of the list

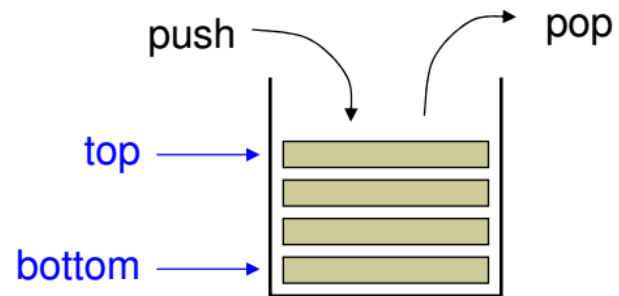


Terminology for Stacks

- **Stack:** A list in which entries are removed and inserted only at the head
- Data are entered into a stack in a Last-In, First-Out manner (LIFO)
- **Pop:** To remove the entry at the top
- **Push:** To insert an entry at the top

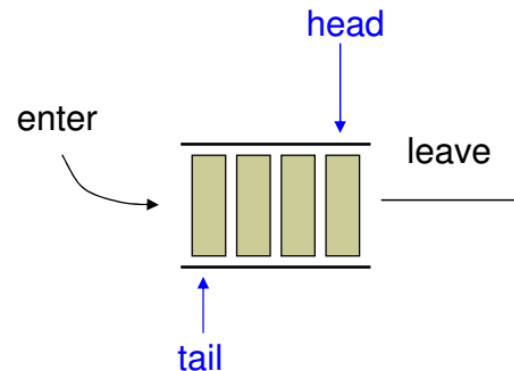
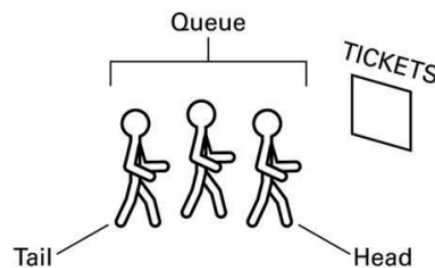


a stack of books



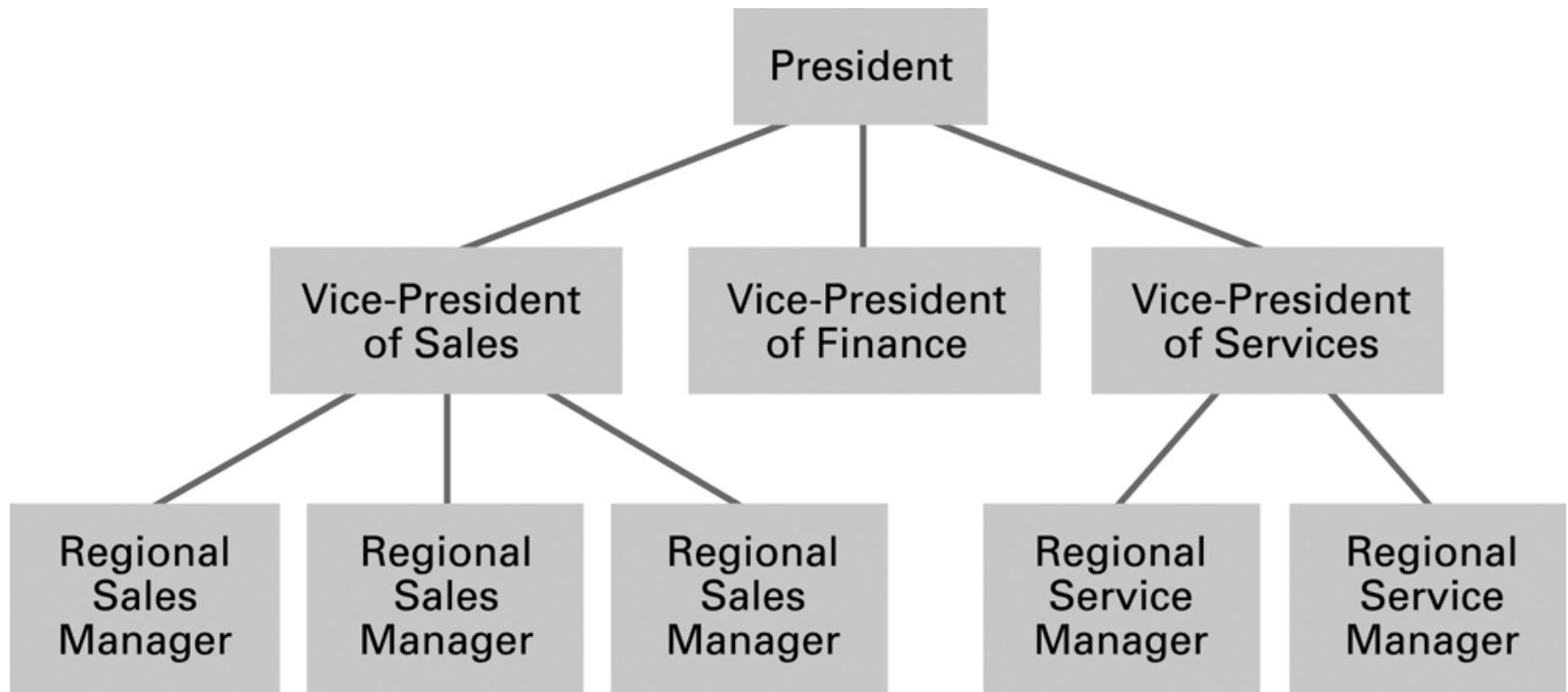
Terminology for Queues

- **Queue:** A list in which entries are removed at the head and are inserted at the tail
- Data are entered into a queue in a first-in, first-out (FIFO) manner.



6/32

An example of an organization chart



Terminology for a Tree

- **Tree:** A collection of data whose entries have a hierarchical organization
- **Node:** An entry in a tree
- **Root node:** The node at the top of the tree
- **Terminal or leaf node:** A node at the bottom of the tree

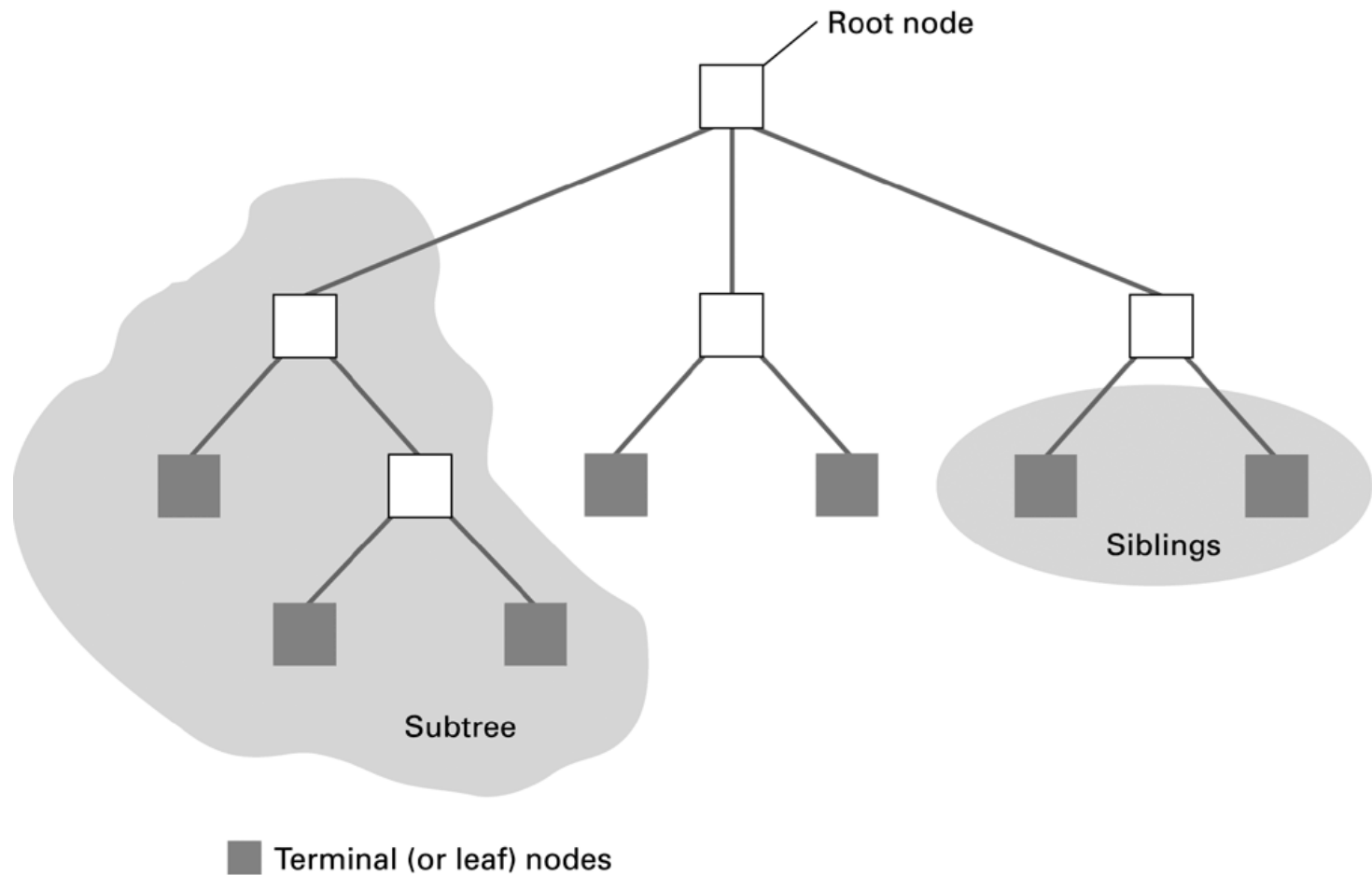
Terminology for a Tree (continued)

- **Parent:** The node immediately above a specified node
- **Child:** A node immediately below a specified node
- **Ancestor:** Parent, parent of parent, etc.
- **Descendent:** Child, child of child, etc.
- **Siblings:** Nodes sharing a common parent

Terminology for a Tree (continued)

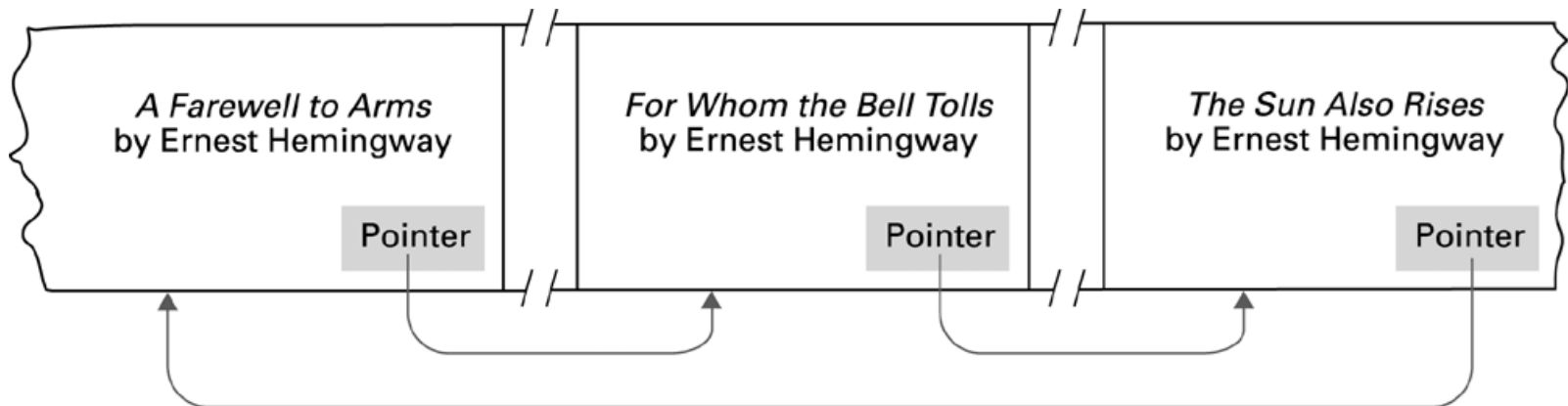
- **Binary tree:** A tree in which every node has at most two children
- **Depth:** The number of nodes in longest path from root to leaf

Tree terminology



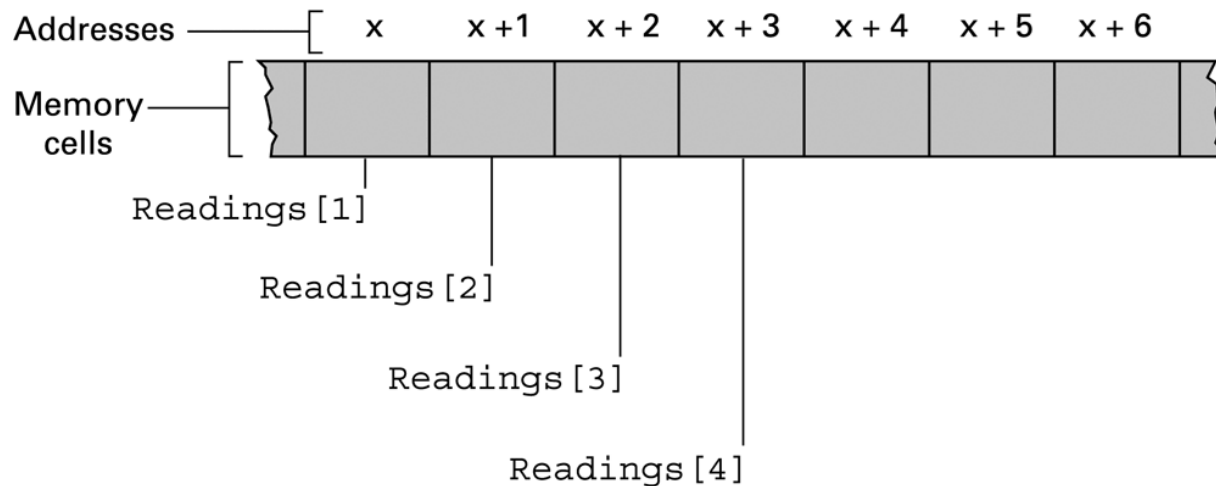
Characteristics of Data Structures

- **Static Data Structures:** Size and shape of data structure does not change
- **Dynamic Data Structures:** Size and shape of data structure can change
- **Pointers:** The address of a data item in memory cell which is used to locate it.



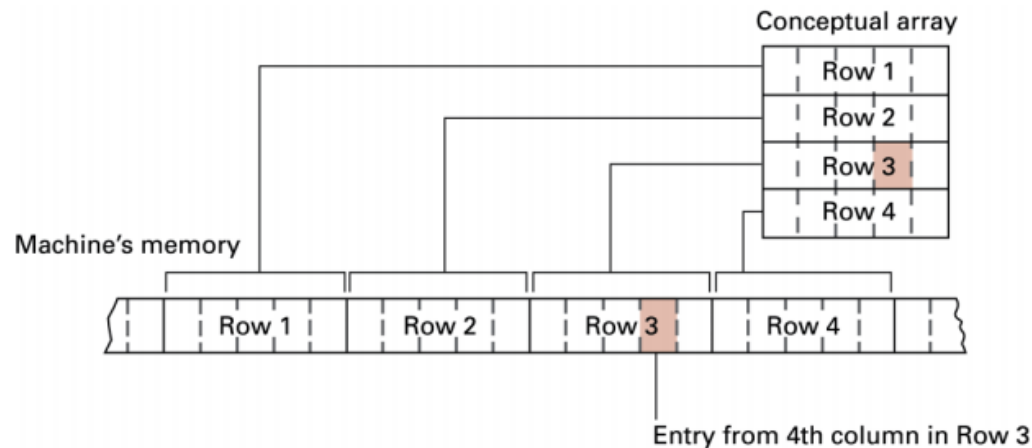
Storing One-Dimensional Arrays

- One-dimensional homogeneous arrays
 - x is initial memory address
 - i is the index of array to be accessed
 - i 'th index is located at memory address $x+(i-1)$



Storing Two-Dimensional Arrays

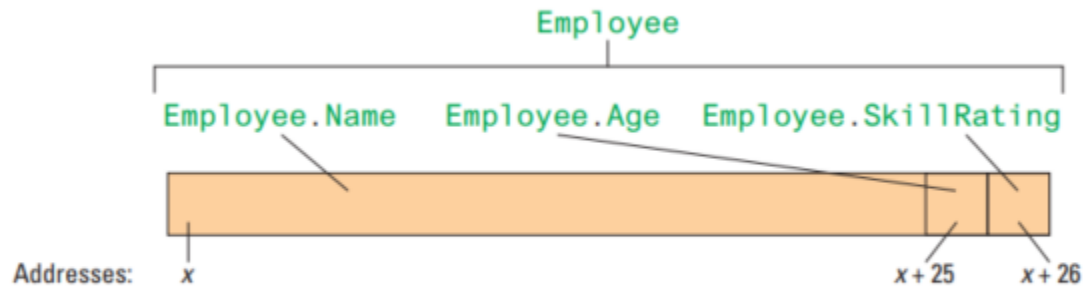
- ❑ Memory cells are in one-dimensional order, to store a two-dimensional array, some mapping must be done:
 - **Row-major:** data entries are stored row-by-row
 - Address polynomial: $A[i, j] = \chi + c \times (i-1) + (j-1)$



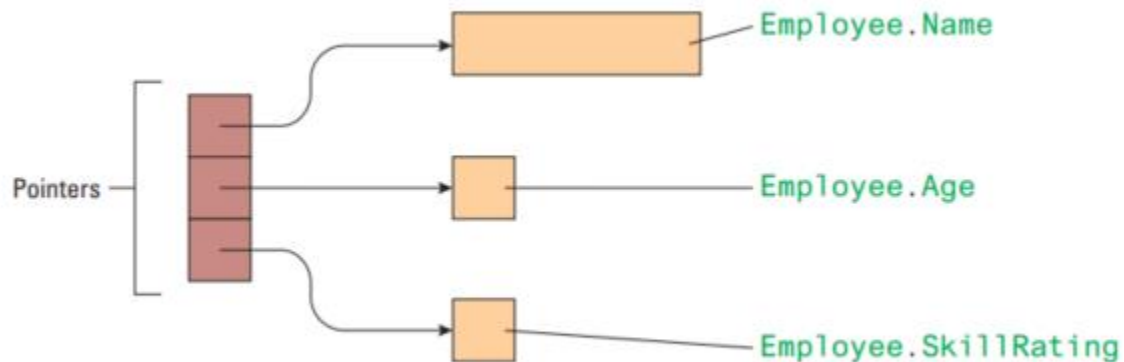
- **Column-major:** data entries are stored column-by-column
 - Address polynomial: $A[i, j] = m + (i-1) + r \times (j-1)$

Note: m is the address of the first data item, i is the row index, j is the column index, c is the number of columns, and r is the number of rows

Storing Aggregates



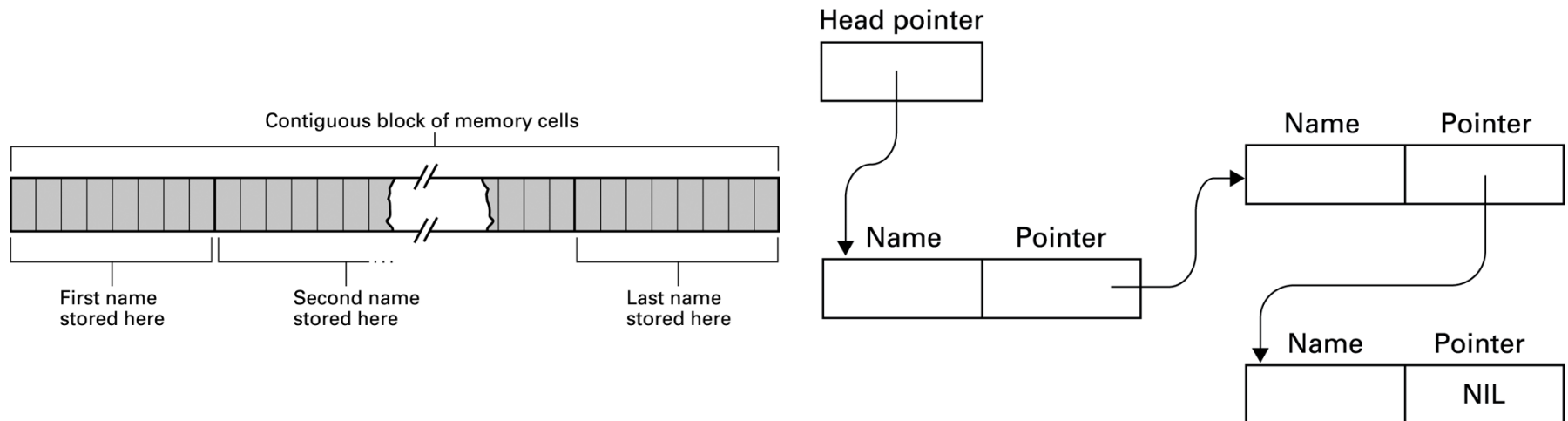
a. Aggregate stored in a contiguous block



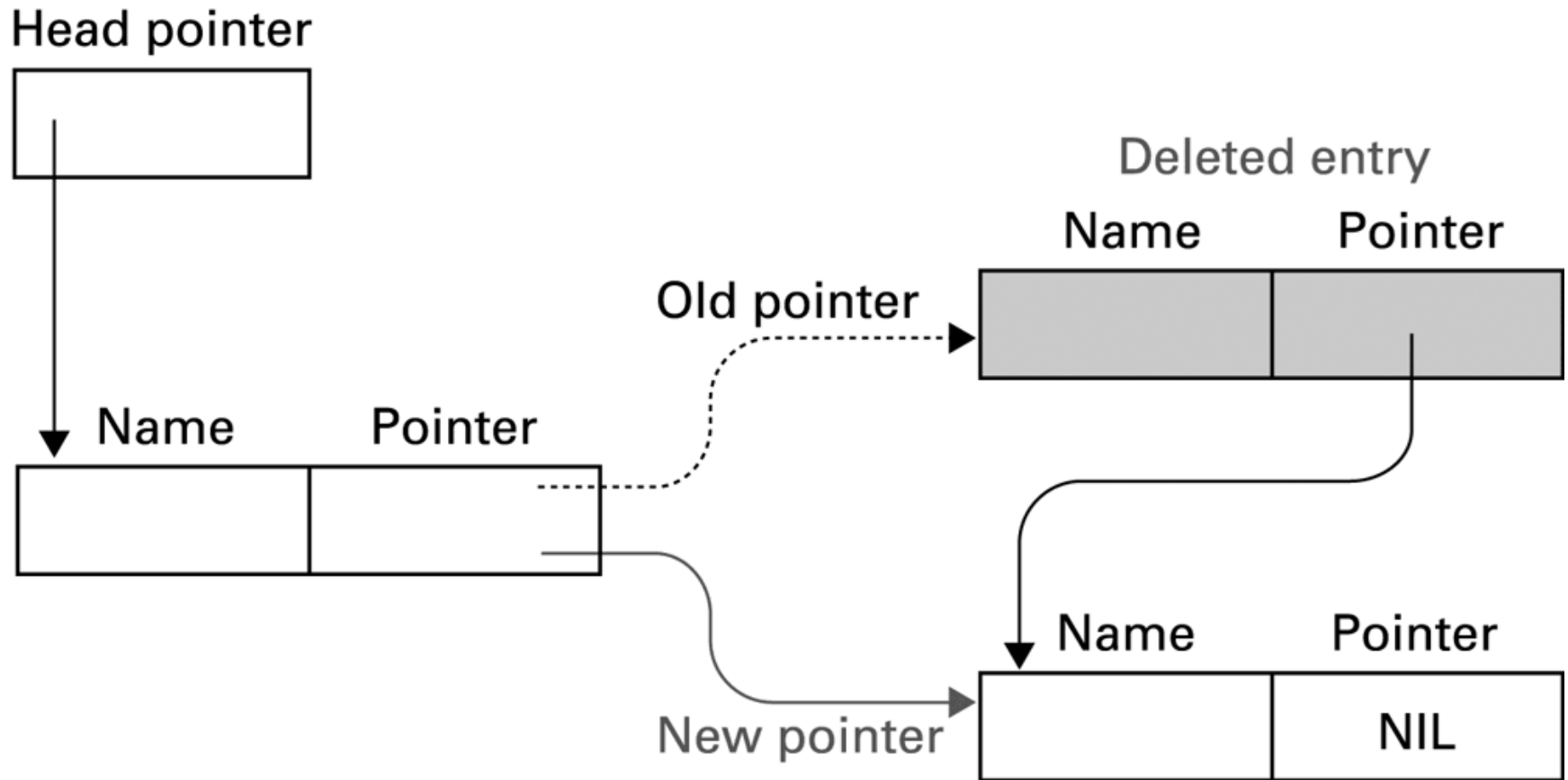
b. Aggregate fields stored in separate locations

Storing Lists

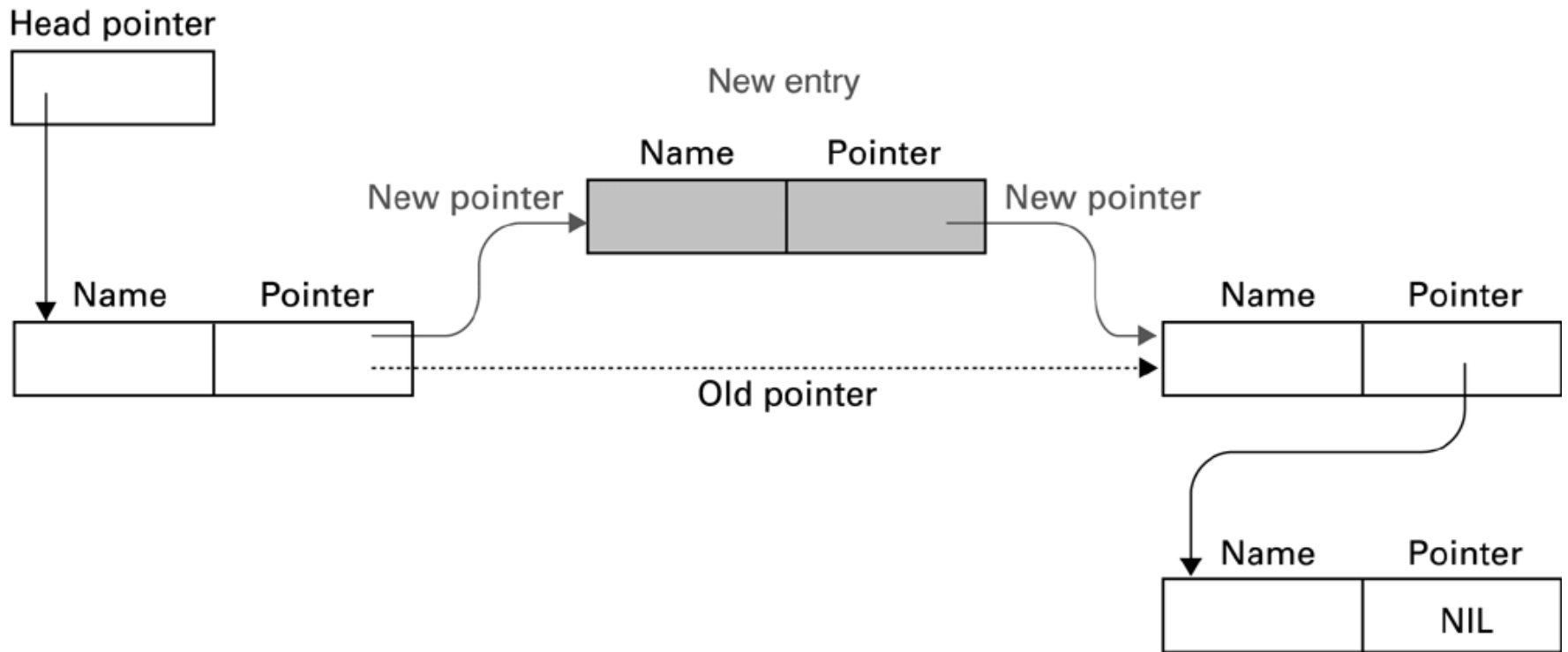
- **Contiguous list:** List stored in a homogeneous array
- **Linked list:** List in which each entries are linked by pointers
 - **Head pointer:** Pointer to first entry in list
 - **NIL pointer:** A “non-pointer” value used to indicate end of list



Deleting an entry from a linked list



Inserting an entry into a linked list



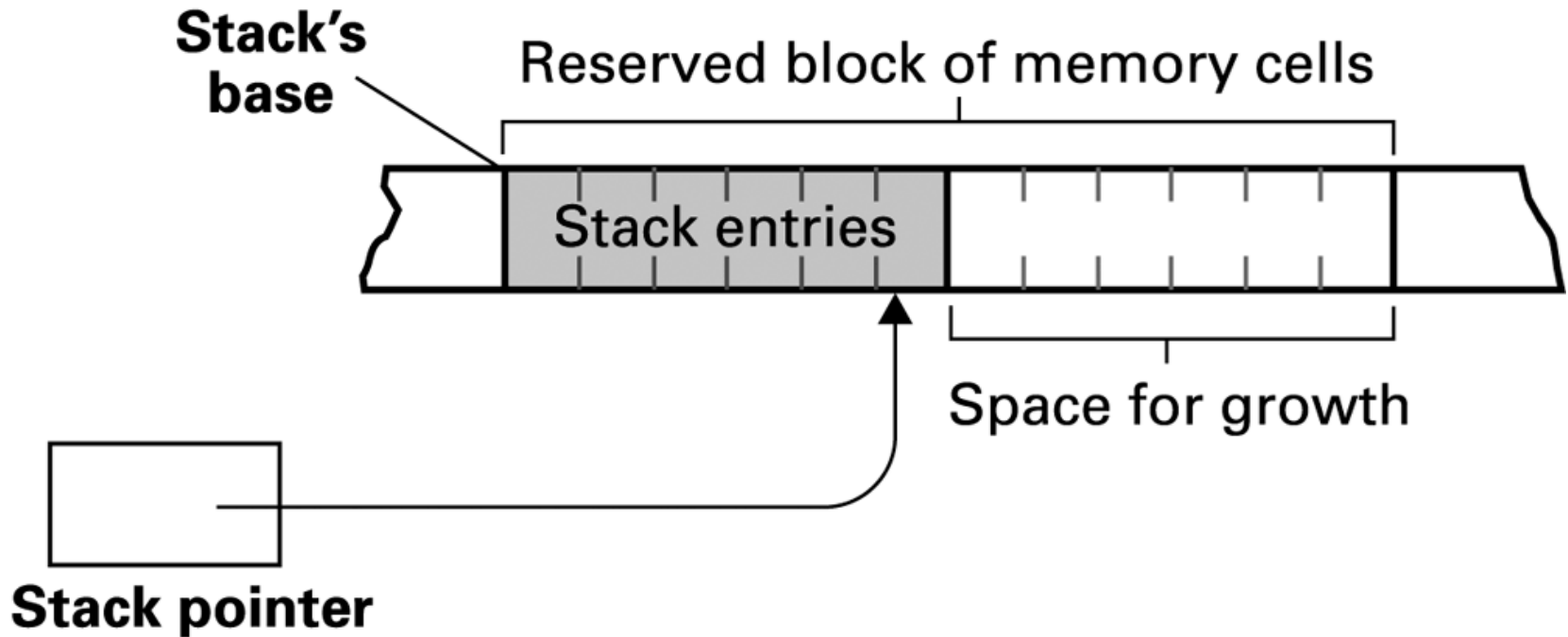
A procedure for printing a linked list

```
def PrintList(List):  
    CurrentPointer = List.Head  
    while (CurrentPointer != None):  
        print(CurrentPointer.Value)  
        CurrentPointer = CurrentPointer.Next
```

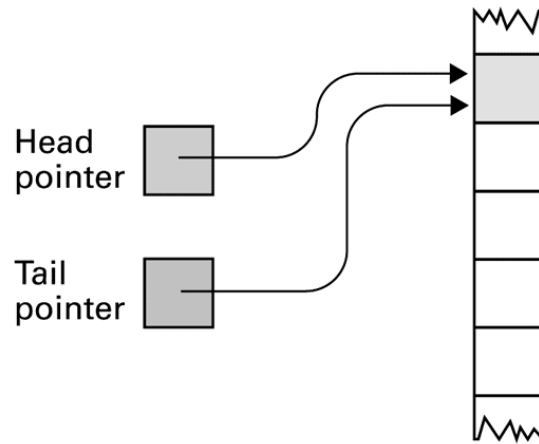
Storing Stacks and Queues

- Stacks usually stored as contiguous lists
- Queues usually stored as **Circular Queues**
 - Stored in a contiguous block in which the first entry is considered to follow the last entry
 - Prevents a queue from crawling out of its allotted storage space

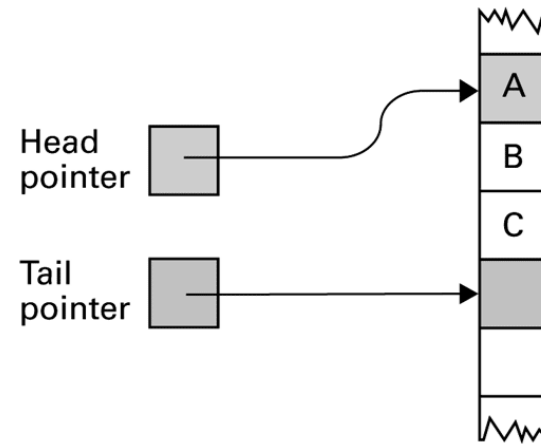
A stack in memory



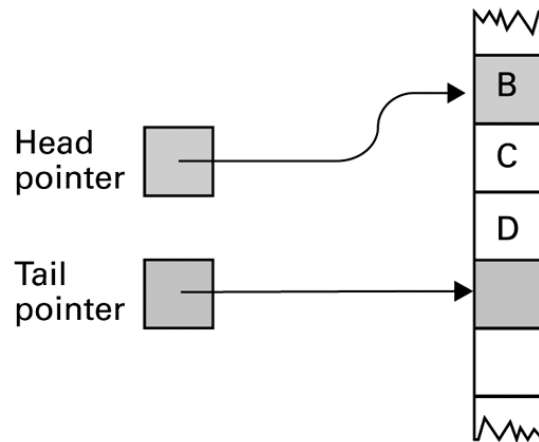
A queue implementation with head and tail pointers



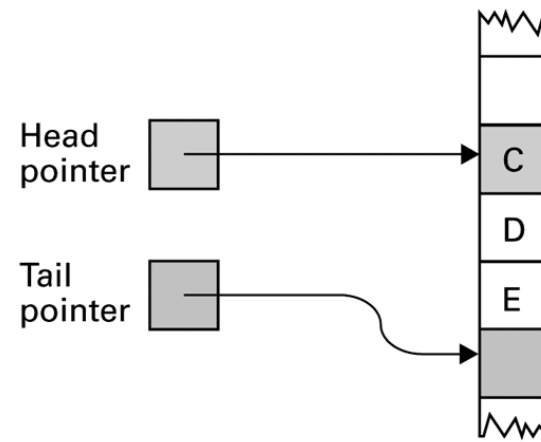
a. Empty queue



b. After inserting entries A, B, and C

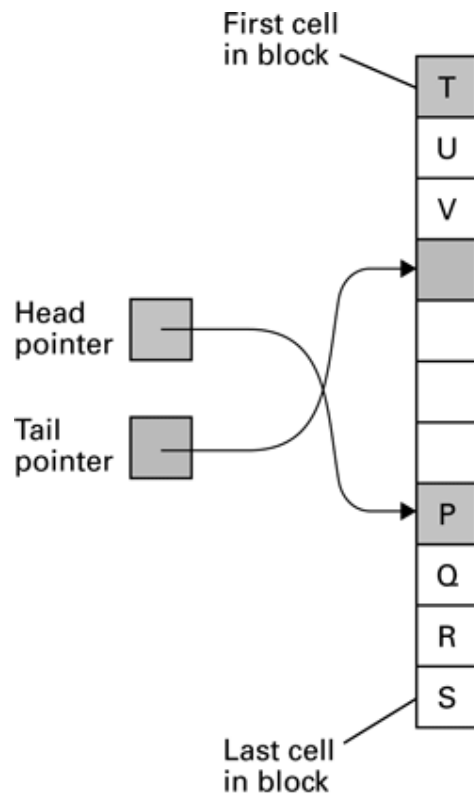


c. After removing A and inserting D

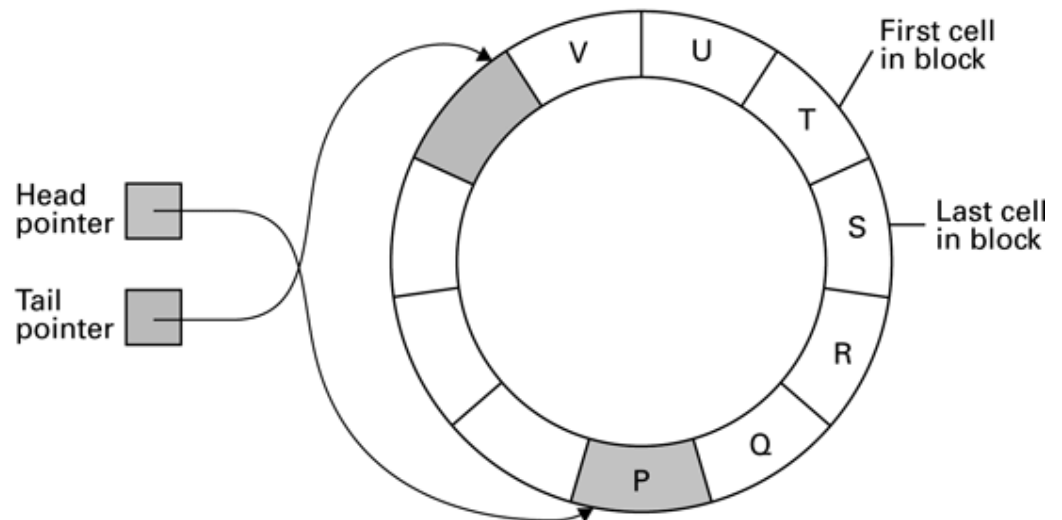


d. After removing B and inserting E

A circular queue containing the letters P through V



a. Queue as actually stored



b. Conceptual storage with last cell "adjacent" to first cell

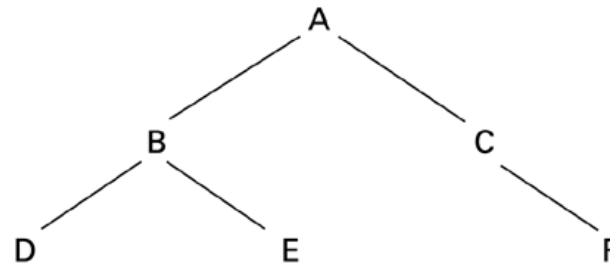
Storing Binary Trees

- Linked structure
 - Each node = data cells + two child pointers
 - Accessed via a pointer to root node
- Contiguous array structure
 - $A[1]$ = root node
 - $A[2], A[3]$ = children of $A[1]$
 - $A[4], A[5], A[6], A[7]$ = children of $A[2]$ and $A[3]$

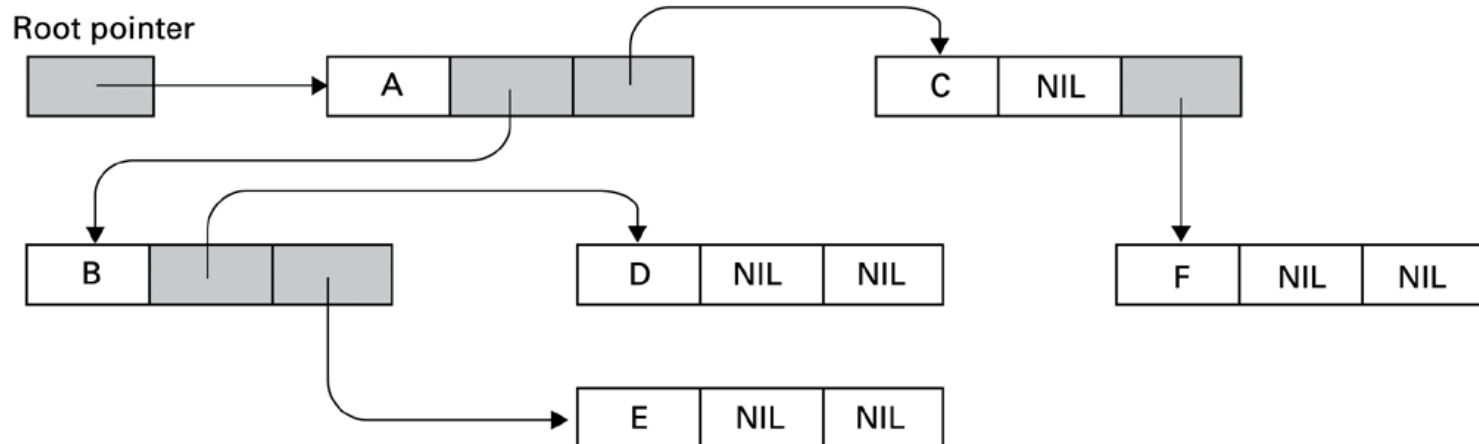
Cells containing the data	Left child pointer	Right child pointer
------------------------------	-----------------------	------------------------

The conceptual and actual organization of a binary tree using a linked storage system

Conceptual tree

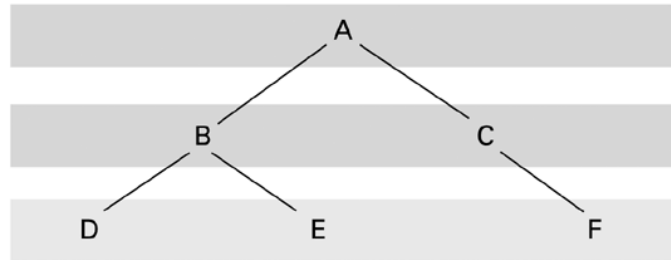


Actual storage organization

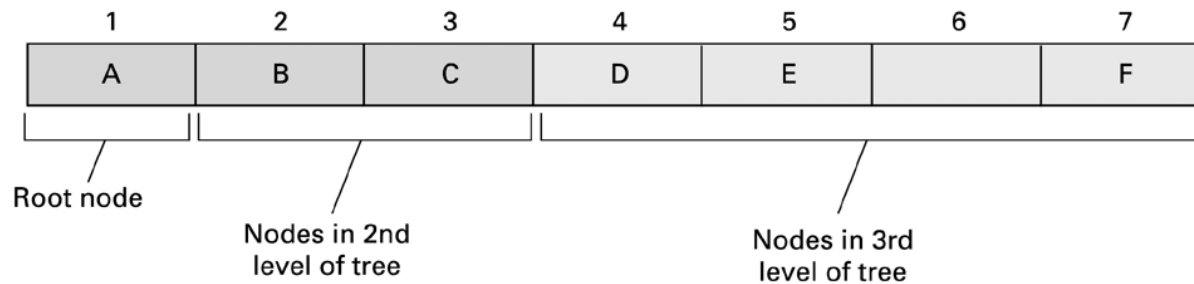


A tree stored without pointers using segmented storage system

Conceptual tree



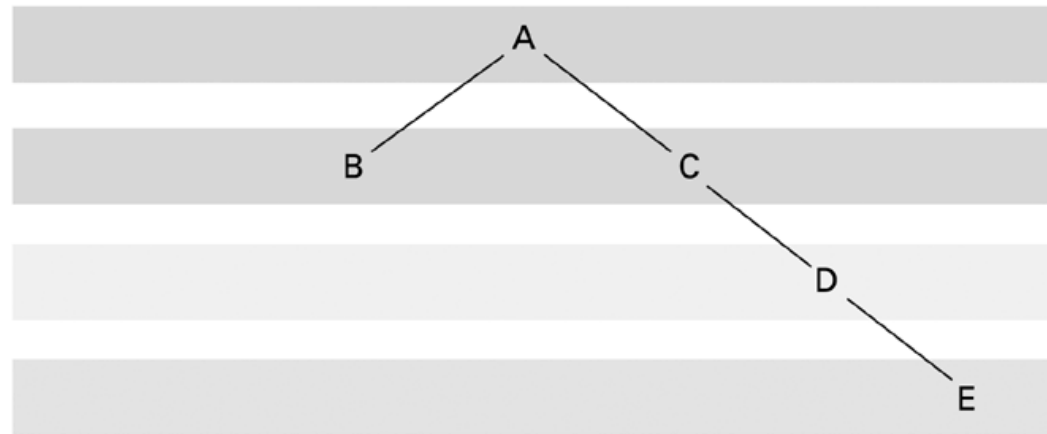
Actual storage organization



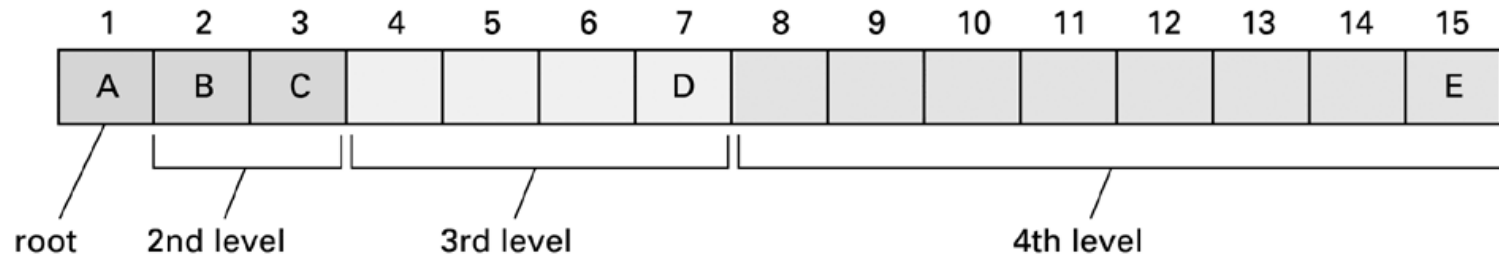
- Left child of node of i is $2i$, and right child $2i + 1$
- Parent node of i is $\lfloor i/2 \rfloor$
- Siblings: if i is even, +1; otherwise -1.

A sparse, unbalanced tree shown in its conceptual form and as it would be stored without pointers

Conceptual tree



Actual storage organization

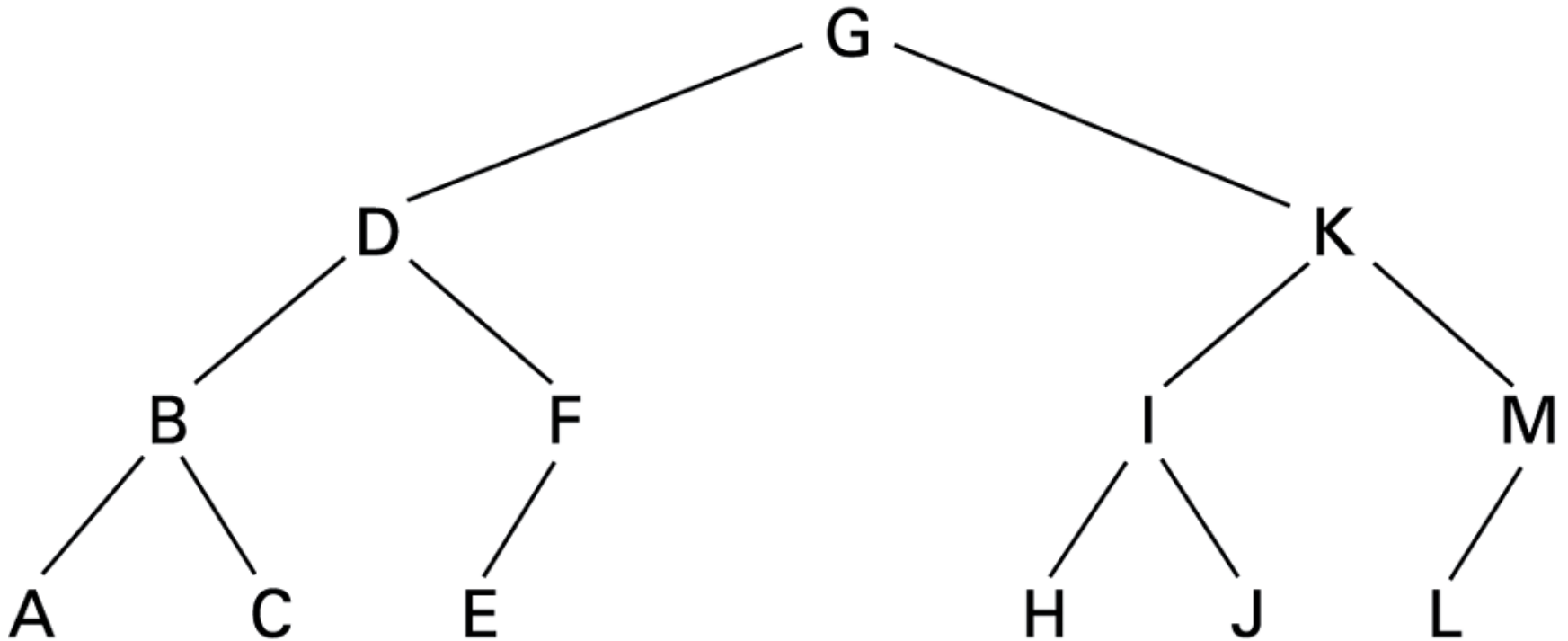


Exercise

Problem:

Construct a data structure consisting of a list of names in alphabetical order along with the operations **search**, **print**, and **insert**.

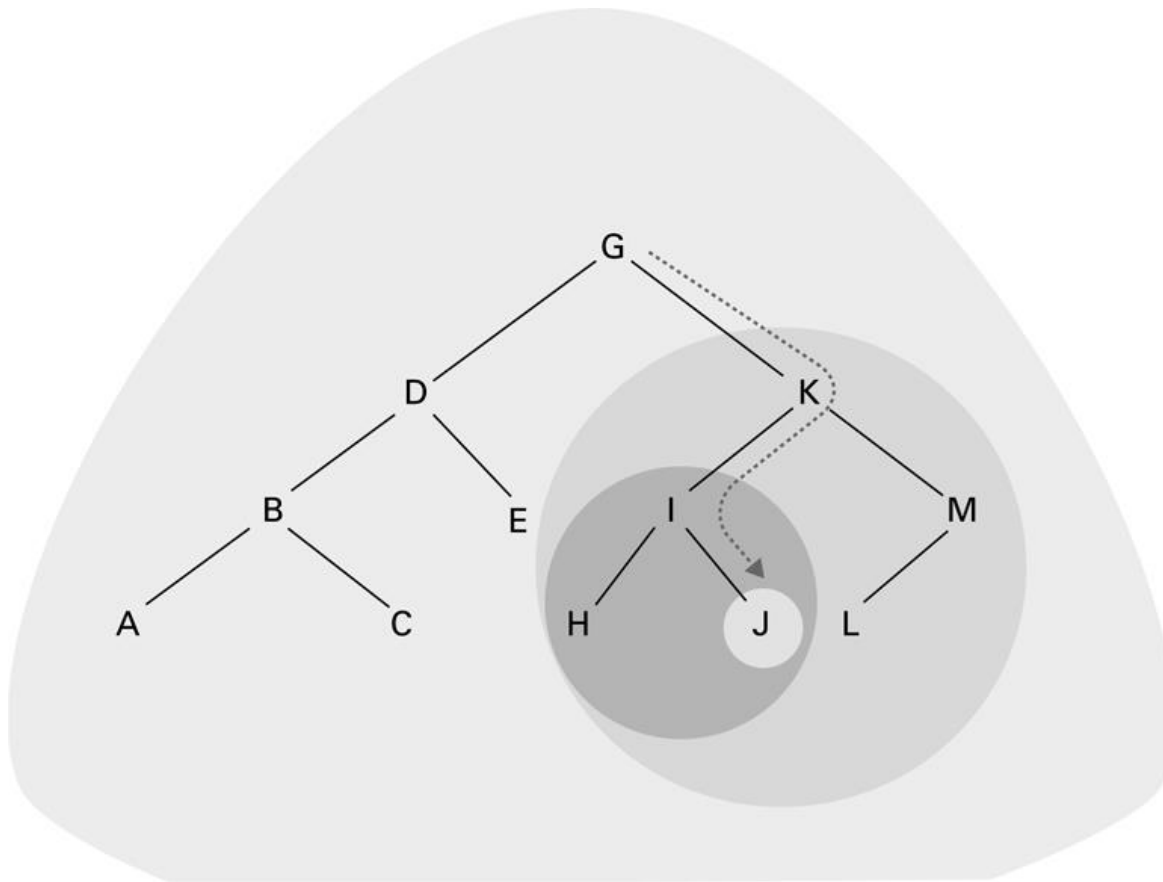
The letters A through M arranged in an ordered tree



Adapted binary search algorithm

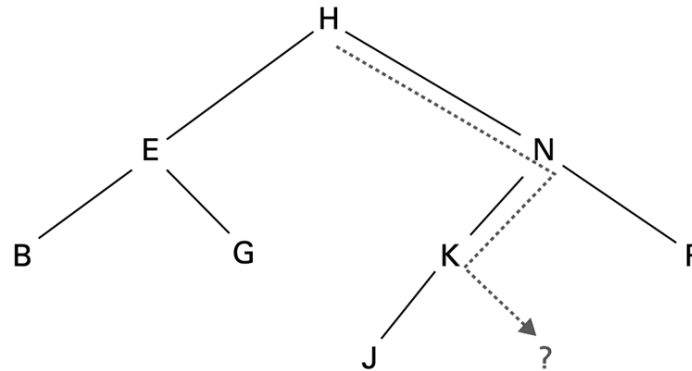
```
def bin_search(node, targetValue):  
    if node is None: return False  
    testEntry = node.data  
    if (targetValue == testEntry):  
        return True  
    if (targetValue < testEntry):  
        return TreeNode.bin_search(node.left, targetValue)  
    if (targetValue > testEntry):  
        return TreeNode.bin_search(node.right, targetValue)
```

Searching for the letter J

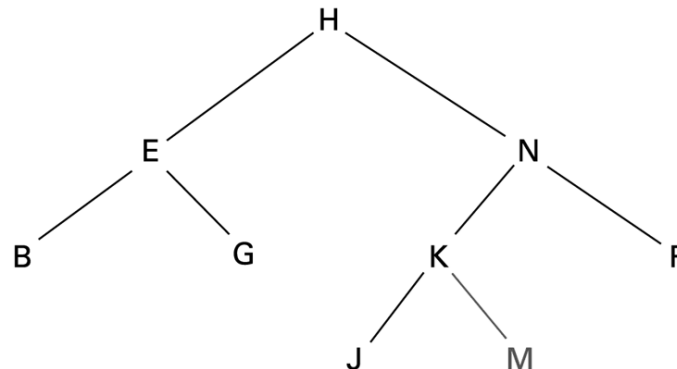


Inserting the entry M into the list B, E, G, H, J, K, N, P stored as a tree

a. Search for the new entry until its absence is detected



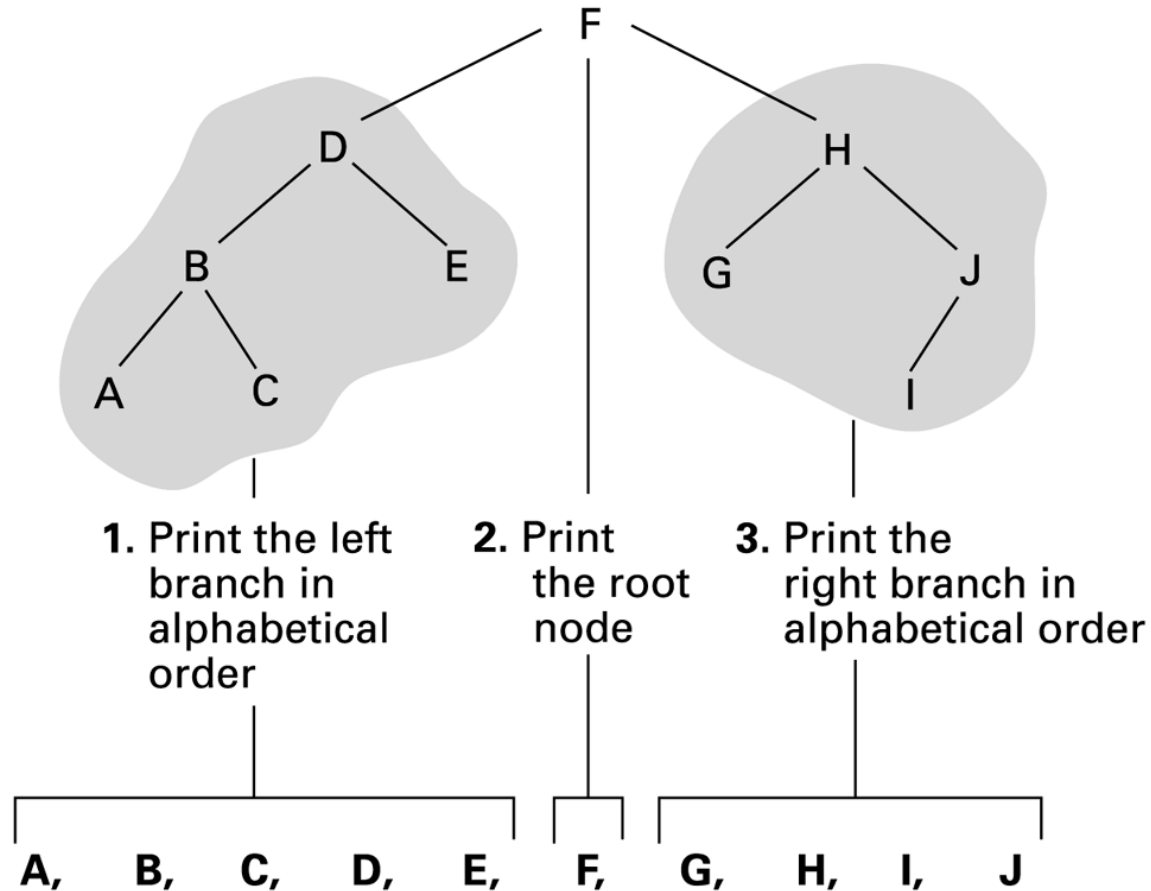
b. This is the position in which the new entry should be attached



A procedure for inserting a new entry in a list stored as a binary tree

```
def insert(node, val):  
    if (node is None):  
        node = TreeNode(val)  
    elif val < node.data:  
        node.left = TreeNode.insert(node.left, val)  
    else:  
        node.right = TreeNode.insert(node.right, val)  
    return node
```

Printing a search tree in alphabetical order



Tree Traversals: In-Order Traversal

- This traversal walks through the tree in an alphabetic order:

```
# in this we traverse first to the leftmost node,  
# then print its data and then traverse for rightmost node  
def inorder(node):  
    if node:  
        TreeNode.inorder(node.getLeft())  
        print(node.getData(), end=' ')  
        TreeNode.inorder(node.getRight())  
    return
```

Other Tree Traversals

*# in this we first print the root node and
then traverse towards leftmost node and then to the rightmost node*

```
def preorder(node):  
    if node:  
        print(node.getData(), end=' ')  
        TreeNode.preorder(node.getLeft())  
        TreeNode.preorder(node.getRight())  
    return
```

*# in this we first traverse to the leftmost node and
then to the rightmost node and then print the data*

```
def postorder(node):  
    if node:  
        TreeNode.postorder(node.getLeft())  
        TreeNode.postorder(node.getRight())  
        print(node.getData(), end=' ')  
    return
```

Suggested Reading/Activity

- Check the visualizations of different data structures at visualgo.net